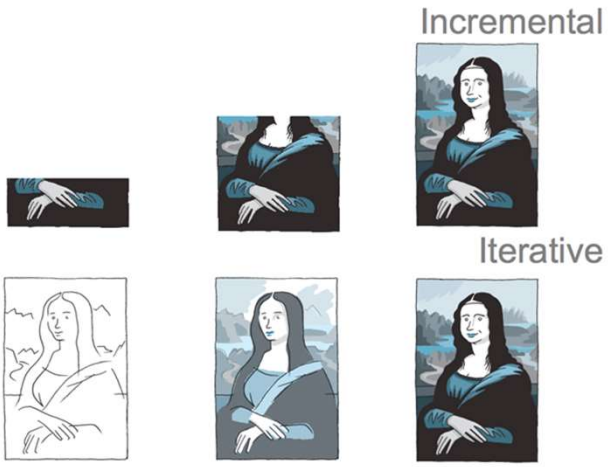
7. Agile Software Development
Embrace change

"Software Development Methodologies"
Ricardo Filipe, ISEC (ricardo.filipe@isec.pt)

Motivation

- https://www.youtube.com/watch?v=5RocT_OdQcA

Motivation



172

172

Motivation

- Incremental



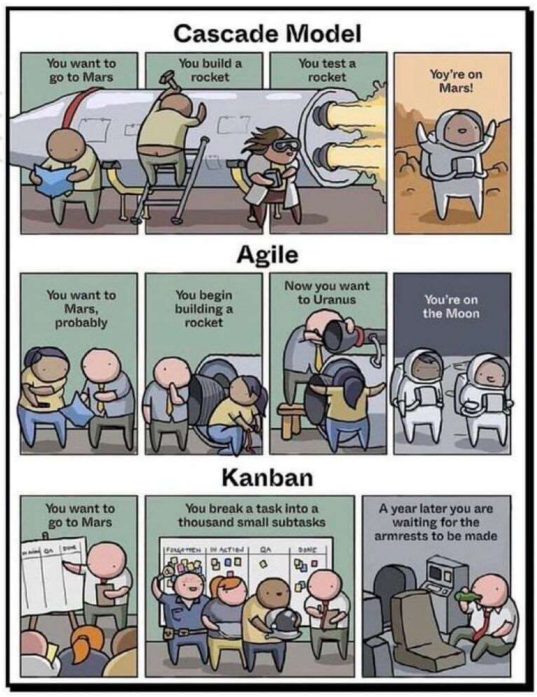
- Iterative

What about Agile?

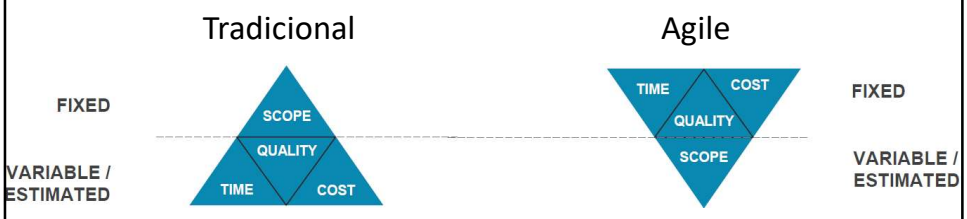


173

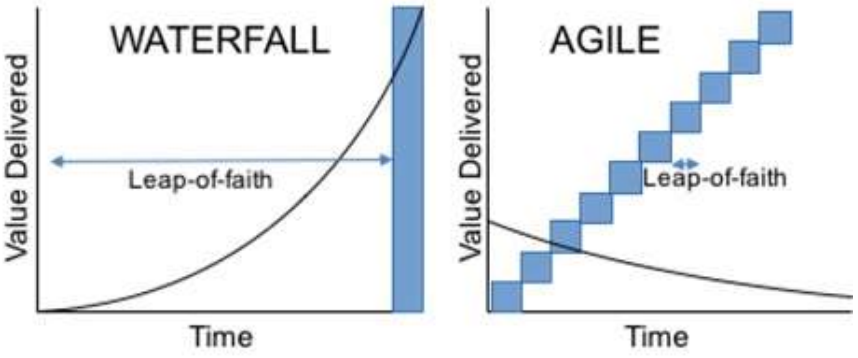
173


Cascade Model
You want to go to Mars
You build a rocket
You test a rocket
Yoy're on Mars!
Agile
You want to Mars, probably
You begin building a rocket
Now you want to Uranus
You're on the Moon
Kanban
You want to go to Mars
You break a task into a thousand small subtasks
A year later you are waiting for the armrests to be made

Motivation


Tradicional
Agile
FIXED
VARIABLE / ESTIMATED
SCOPE
QUALITY
TIME
COST
TIME
COST
QUALITY
SCOPE
FIXED
VARIABLE / ESTIMATED

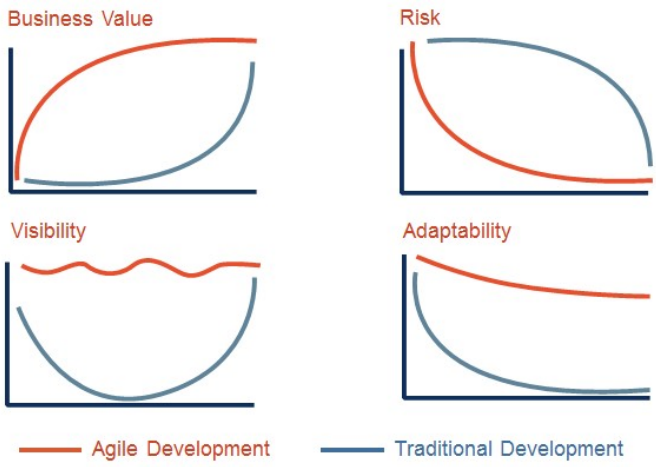
Motivation



176

176

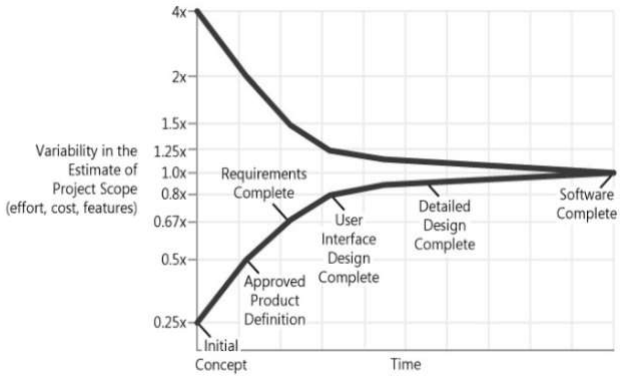
Motivation



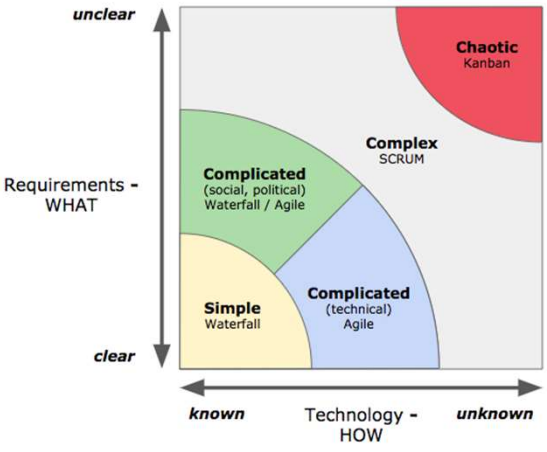
177

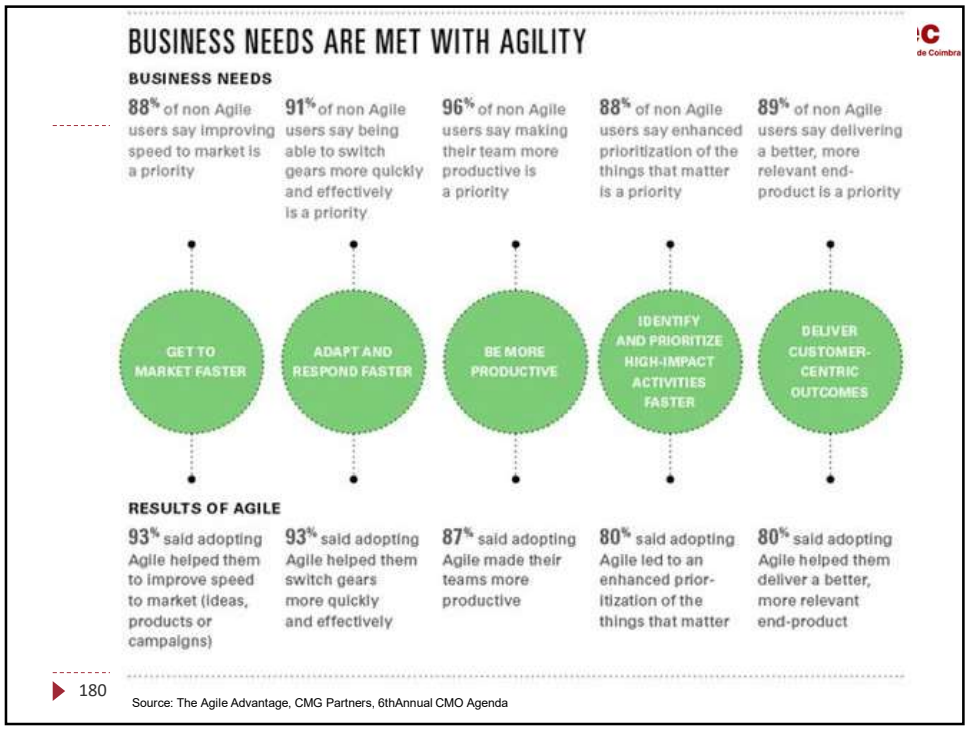
177

Motivation



Motivation





180

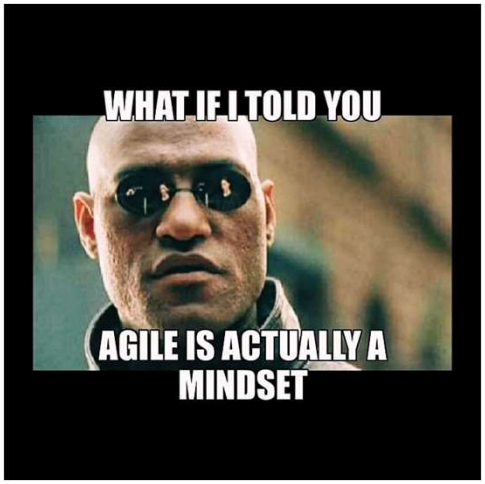
Source: The Agile Advantage, CMG Partners, 6thAnnual CMO Agenda

Motivation



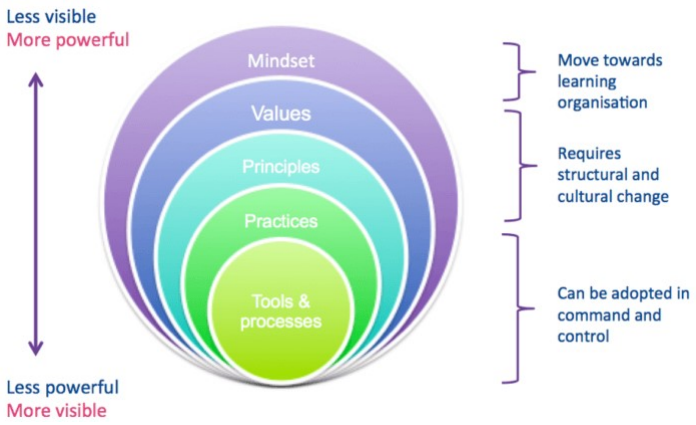
181

Motivation

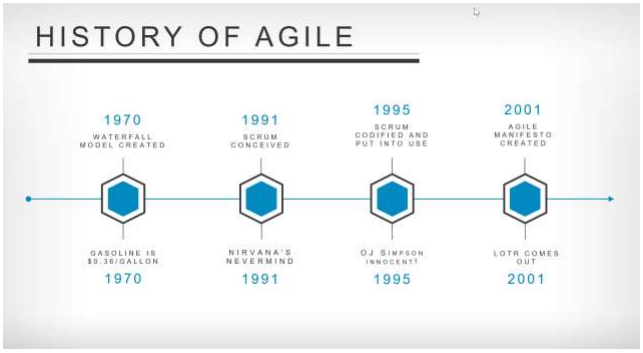


©2008-2021 João Cunha / Ricardo Filipe, DEIS-ISEC

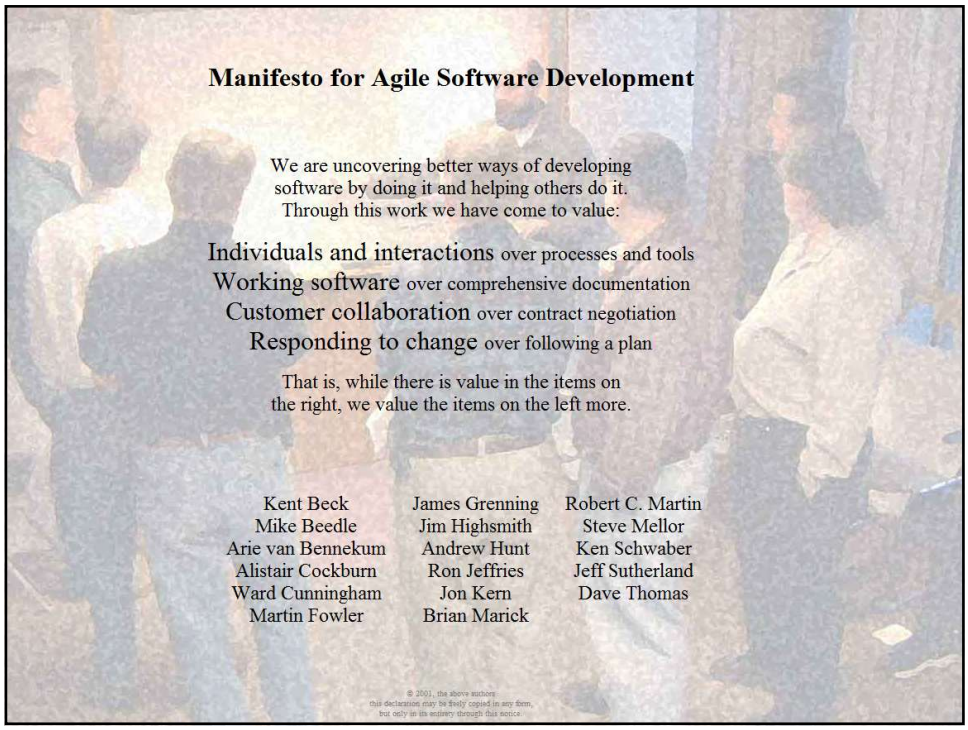
Agile is not only tools and processes



History of Agile



https://the-agilesphere.com/2016/09/18/the-history-of-agile/



The Agile Manifesto

- Published in 2001 by the Agile Alliance

Individuals and Interactions	over processes and tools
Working Software	over comprehensive documentation
Customer Collaboration	over contract negotiation
Responding to Change	over following a plan

  - while there is value in the items on the right, we value the items on the left more

http://agilemanifesto.org/

186

Manifesto para o Desenvolvimento Ágil de Software.

Ao desenvolver e ao ajudar outros a desenvolver software,
temos vindo a descobrir melhores formas de o fazer.
Através deste processo começámos a valorizar:

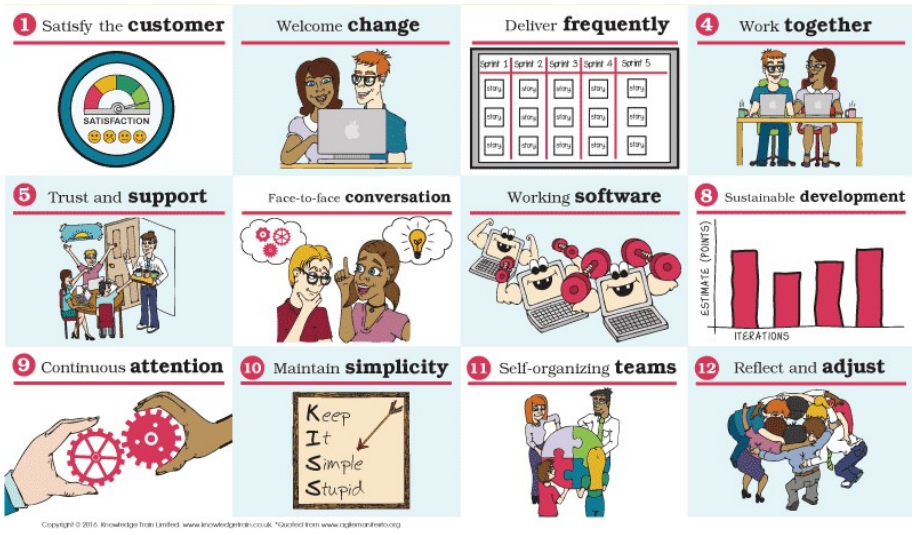
Indivíduos e interacções mais do que processos e ferramentas
Software funcional mais do que documentação abrangente
Colaboração com o cliente mais do que negociação contratual
Responder à mudança mais do que seguir um plano

Ou seja, apesar de reconhecermos valor nos itens à direita,
valorizamos mais os itens à esquerda.

Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas

© 2001, os autores acima indicados
Esta declaração pode ser copiada e distribuída livremente, e de qualquer forma,
mas apenas se for feita de forma integral.

187



188

188

The Agile Principles (1)

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter time scale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

189

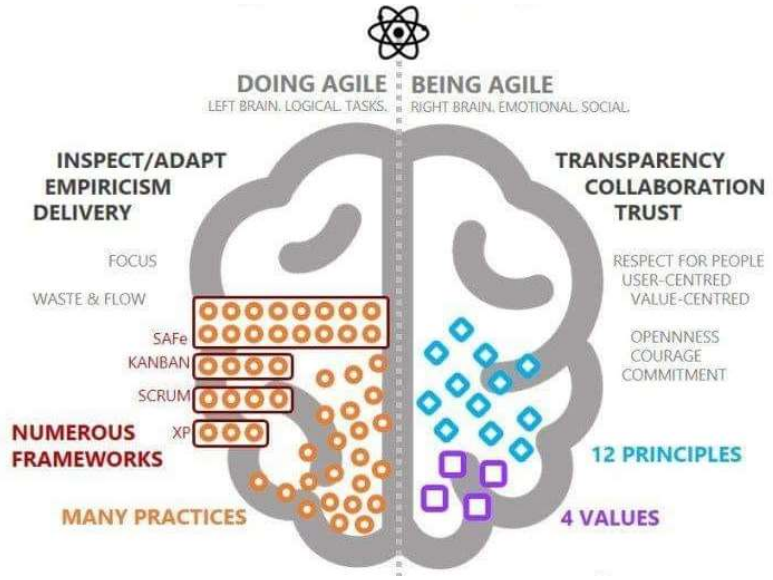
189

The Agile principles (2)

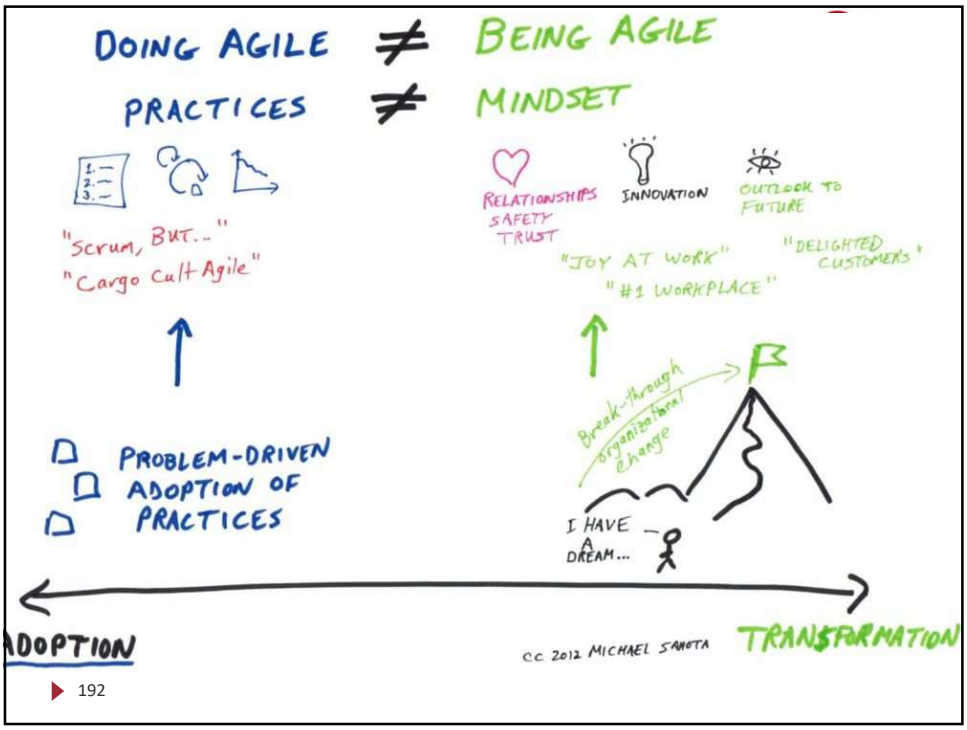
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

190

Agile Mindset

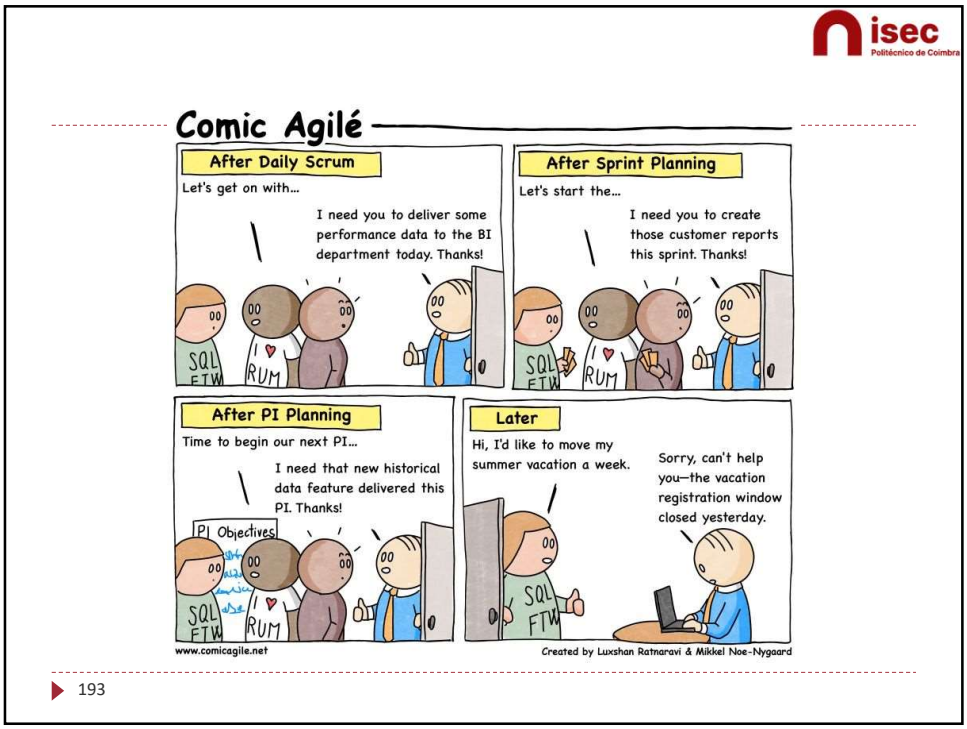


191



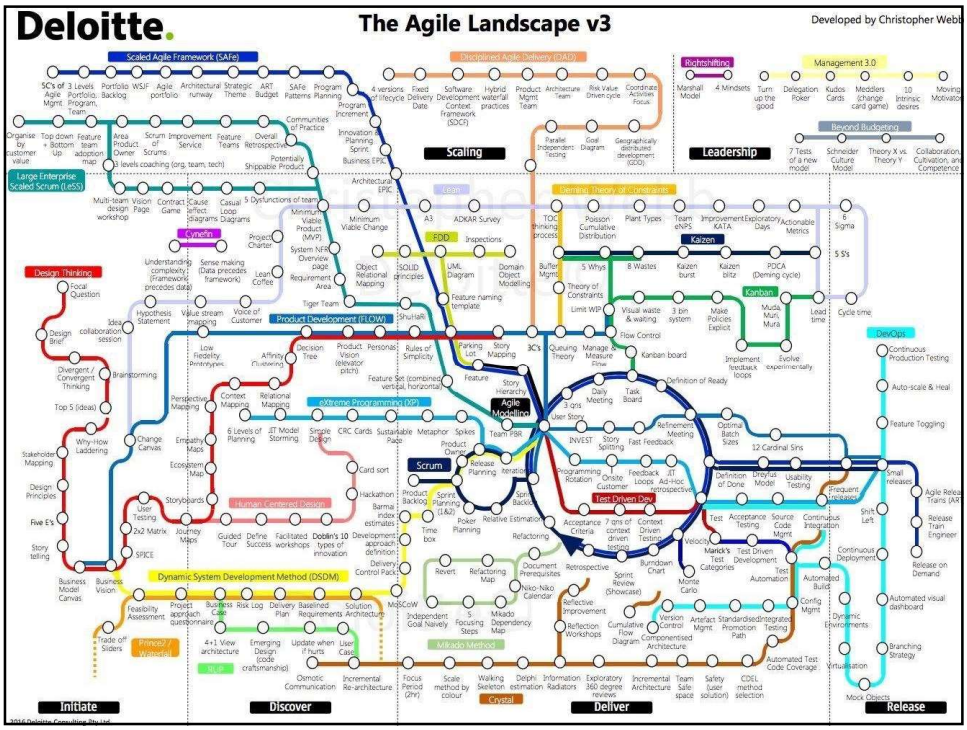
192

192

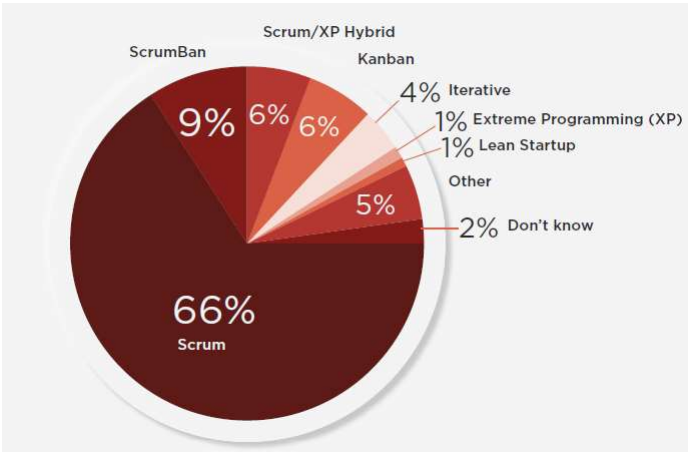


193

193

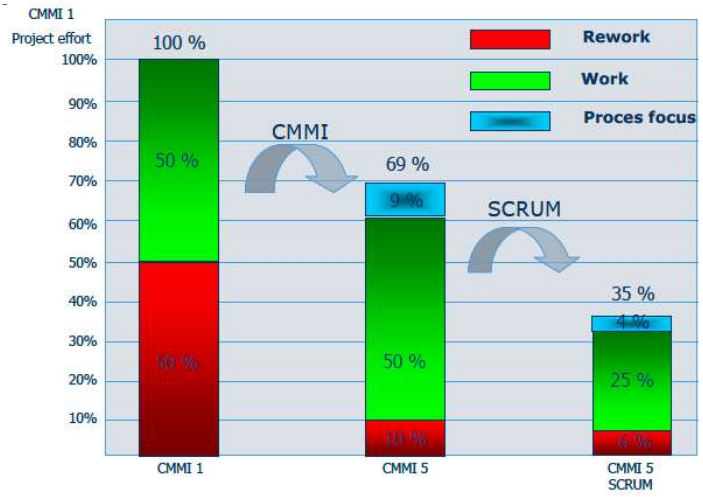

Deloitte.
The Agile Landscape v3
Developed by Christopher Webb
Scaled Agile Framework (SAFe)
Disciplined Agile Delivery (DAD)
Scaling
Leadership
Large Enterprise Scaled Scrum (LeSS)
Design Thinking
Product Development (FLOW)
eXtreme Programming (XP)
Scrum
Agile Modelling
Kaizen
Kanban
Lean
Test Driven Dev
Dynamic System Development Method (DSDM)
Human Centered Design
Mikado Method
Crystal
DevOps
Initiate
Discover
Deliver
Release

194


Scrum/XP Hybrid
ScrumBan
Kanban
4% Iterative
1% Extreme Programming (XP)
1% Lean Startup
Other
5%
2% Don't know
9%
6%
6%
66%
Scrum

195

195



196

196

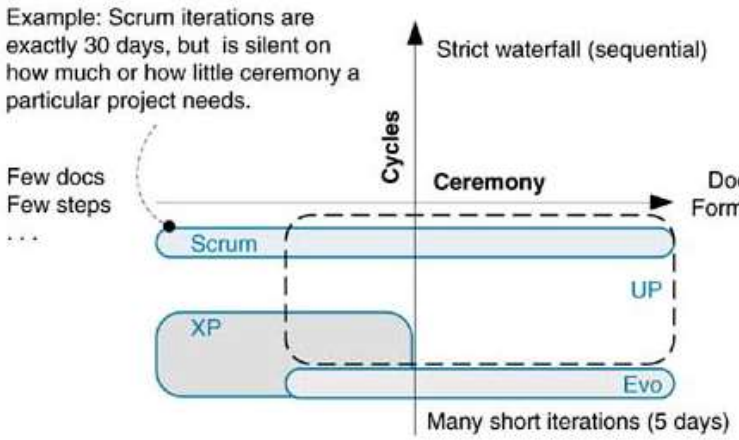
What makes a method Agile?

- Short iterations
- Incremental design
- User involvement
- Minimal documentation
- Informal communication
- Adaptive planning
- Evolutionary delivery

197

197

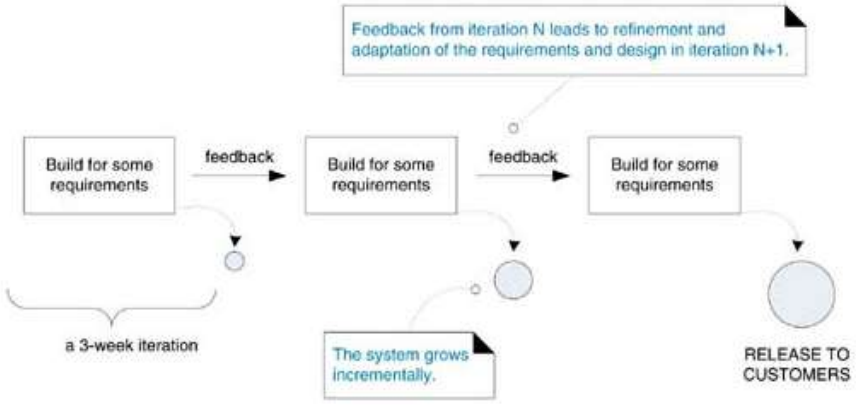
Agile methods classification



198

198

Iterative and incremental development

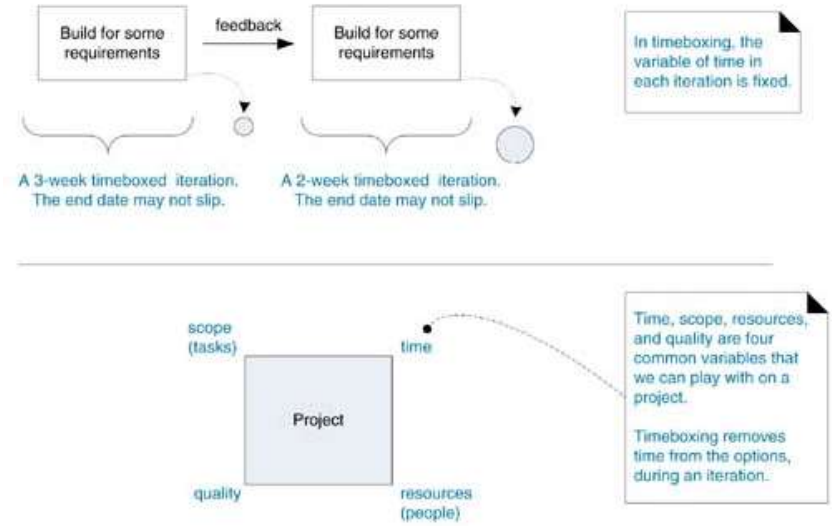


- No changes from external stakeholders during the iteration

199

199

Timeboxed development

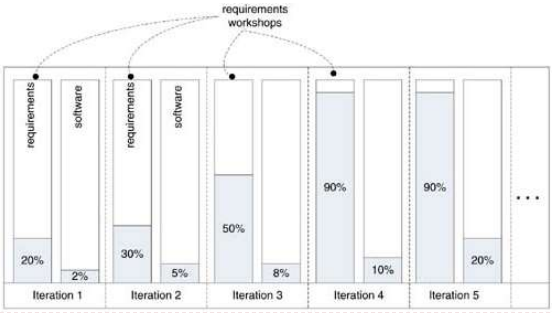


200

200

Evolutionary and adaptive (1)

- Development
  - Requirements, plan, estimates, and solution evolve or are refined over the course of the iterations
    - Adaptive: implies a feedback, from prior work
- Requirements analysis
  - Most requirements discovery and refinement during early iterations
  - Early top-ten requirements list

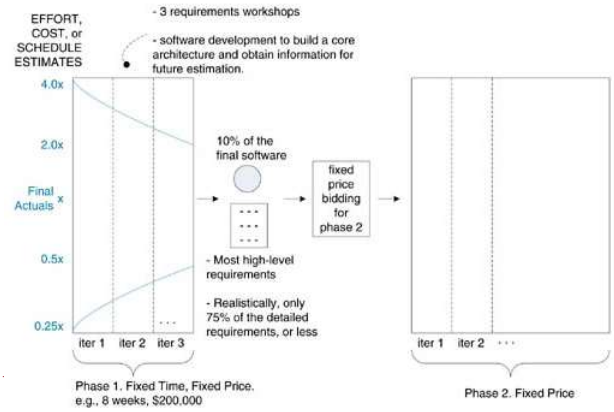


201

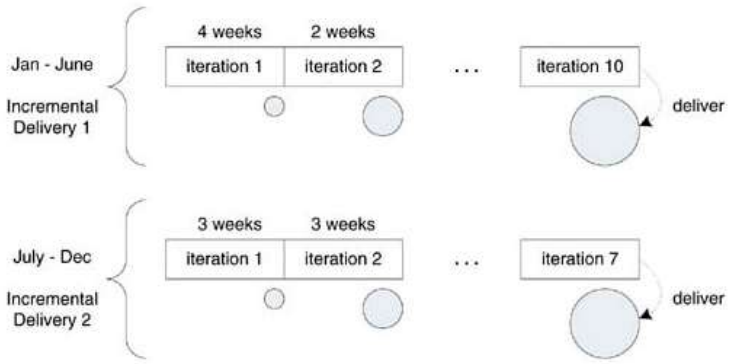
201

Evolutionary and adaptive (2)

- Planning
  - Adaptive rather than predictive
  - Preferred 2 contract phases
  - Risk-driven or client-driven

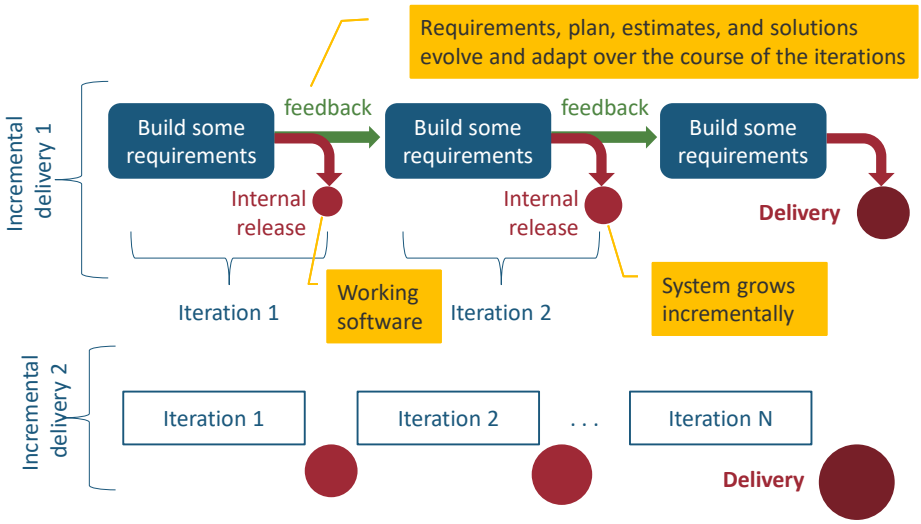


Evolutionary and incremental delivery



- In evolutionary delivery there is no fixed plan of future deliveries

Iterative, evolutionary, adaptive and incremental delivery



204

204

Agile project management principles

1. Deliver something useful to the client; check what they value.
2. Cultivate committed stakeholders.
3. Employ a leadership-collaboration style.
4. Build competent, collaborative teams.
5. Enable team decision making.
6. Use short timeboxed iterations to quickly deliver features.
7. Encourage adaptability.
8. Champion technical excellence.
9. Focus on delivery activities, not process-compliance activities.

[from: Highsmith, J. 2002. Agile Project Management: Principles and Tools. Cutter Consortium Executive Report 4(2)]

205

205

Agile project management practices

1. **Guiding Vision** – Establish a guiding vision for the project and continuously reinforce it through words and actions.
2. **Teamwork & Collaboration** – Facilitate collaboration and teamwork through relationships and community.
3. **Simple Rules** – Establish and support the team's set of guiding practices, such as Scrum or XP.
4. **Open Information** – Provide visible and open access to project management and other information.
5. **Light Touch** – Apply just enough control to foster emergent behavior in a self-directed team.
6. **Agile Vigilance** – Reinforce the vision, follow or adapt the rules, listen to the people.

[from: Augustine, S., and Woodcock, S. 2002. "Agile Project Management: Emergent Order through Visionary Leadership." CC Pace Systems. July 2002]

206

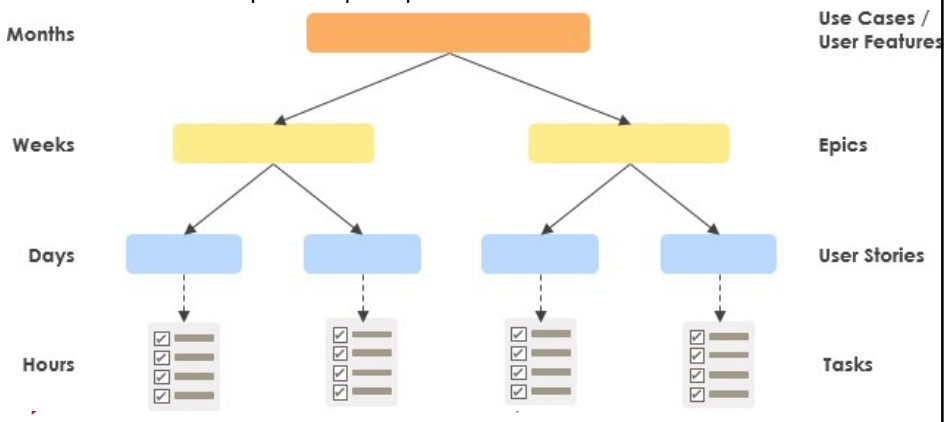
Some agile best practices

- Embrace Communication and Feedback
  - Increase communication, especially face-to-face conversation
- Programming As If People Mattered (AP1, 8)
  - importance of happy developers; avoiding working overtime
- Simple Practices and Project Tools (AP11)
  - do the simplest thing that could possibly work; low-tech, high-touch
- Empirical vs. Defined & Prescriptive Process (AP12,13)
  - An agile method or project lies on a continuum of more or less empirical, driven by need
- Principle-Based versus Rule-Based
  - Agile project management is more than a set of practices—it is a mindset
- Sustainable Discipline: The Human Touch
  - Emphasize human factors: enjoyment, simplicity, short-term reward, peer pressure, …
- Team as a Complex Adaptive System
  - self-organizing teams is more valuable than control or planning by managers

207

Agile Product Backlog

- The backlog for execution.
- Input for planning and iterations
- Maitained by the Product Owner (or Product Manager).
- Follows a decomposition principle



208

Agile Product Backlog

- Might vary definitions regarding the consulted bibliography (epic and feature position on the graph)
- Epic are too large to be complete in one iteration.
- New stories might be added to the backlog.
- User stories prioritization might vary during time.
- Stories might be sliced (if a story is estimated to take more than 1 iteration to finish)

209

209

Agile User Stories

- User stories are short descriptions of what the customer wants the software to do, written by the customer in the customer terminology
- Can be handwritten on paper index cards

As a customer representative, I can search for my customers by their first and last name

A user can view detailed information on a book. For example, number of pages, publication date and a brief description.
Pri: 8
Estimate: 4

A repeat customer must be able to find one book and complete an order in less than 90 seconds.
(Constraint)

210

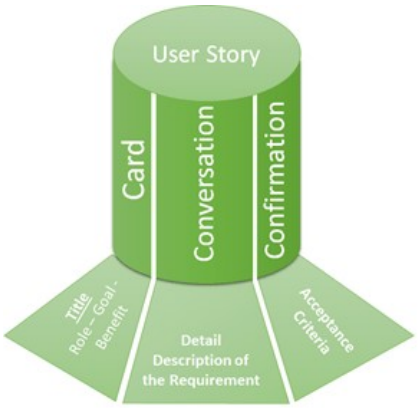
Agile User Stories

- User story is a show description of a specific client requirement:
  - RBG format: Role; goal; benefit
  - As a <type of user> , I want <some goal> so that <some reason>
- Type of user:
  - who are we building this for?
  - What is the persona ?
- Some goal:
  - What is the intention?
  - What are they realy trying to achieve?
- Some reason:
  - What's the overall benefit?
  - What is the big problem that needs to be solved?

211

Agile User Stories

- To write a good user story, you can use the three C's formula:
  - Card
  - Conversation
  - Confirmation



212

212

Agile User Stories

- To write a good user story, you can also check with the acronym **INVEST:**



213

213

Agile User Stories

I
- Independent and self contained

N
- Negotiable and open for collaboration

V
- Valuable to the customer and the organization

E
- Estimable in whatever manner the team estimates

S
- Small enough to be the most accurate when estimating

T
- Testable and specific enough to build tests prior to and after development

Agile User Stories

- Acceptance Criteria:
  - Allows to check if the user story is really “done”.
  - Gathers the requirements and all scenarios that allows to understand that the user story is fully finished.
- To define scenarios
- To achieve consensos
- To serve as a base for testing
- Manage expectations

Agile User Stories

- Acceptance Criteria:

Add Prospect

As a property manager I want to add a new prospect to the lead management system so I can track my interactions with the prospect.

Conditions of Satisfaction

Capture name, email, phone #, contact date, contact format, lease type, and move-in date

Verify prospect is associated with an existing campaign

Agile User Stories

DESCRIPTION:

As a customer, I want to order and pay for the book via a secure web-based form, so that my credit card information is safe.

ACCEPTANCE CRITERIA:

- All mandatory fields must be completed before a customer can submit a form.
- Information from the form is stored in the customer orders database.
- Payment can be made via Amex, Master Card, or Visa credit card.
- The system shall accurately calculate and apply sales tax.
- The system shall accurately calculate and apply shipping charges.
- The customer shall be able to verify the accuracy of the order.
- An acknowledgment email is sent to the customer submitting the form.
- Protection against spam is working.
- The code should be deployed and running in Staging environment.

Agile User Stories Estimation

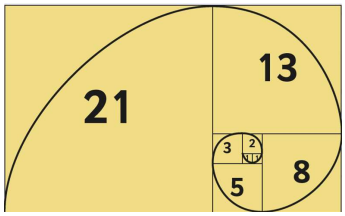
- Requirements are broken “just in time” aka “last responsible moment”
  - Opposite of traditional metodologies
  - Allows to have a more concrete vision of the project and possible risks.

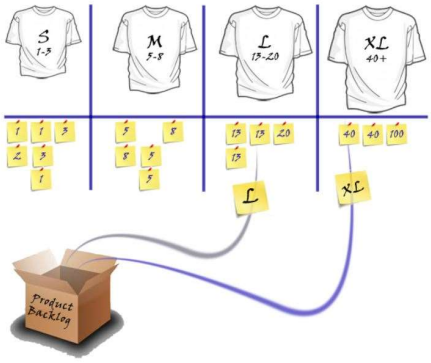
Agile User Stories Estimation

- Estimation based on:
  - Story points
  - Ideal days
- Story points- number of points (effort) to finish a user story
  - Only have meaning inside a project.
- Estimation based on:
  - Complexity
  - Risk
  - Uncertainty
  - Amount of work to do

Agile User Stories Estimation

- Story points estimation normally based on:
  - Fibonacci sequence
  - T-shirt sizing
  - …



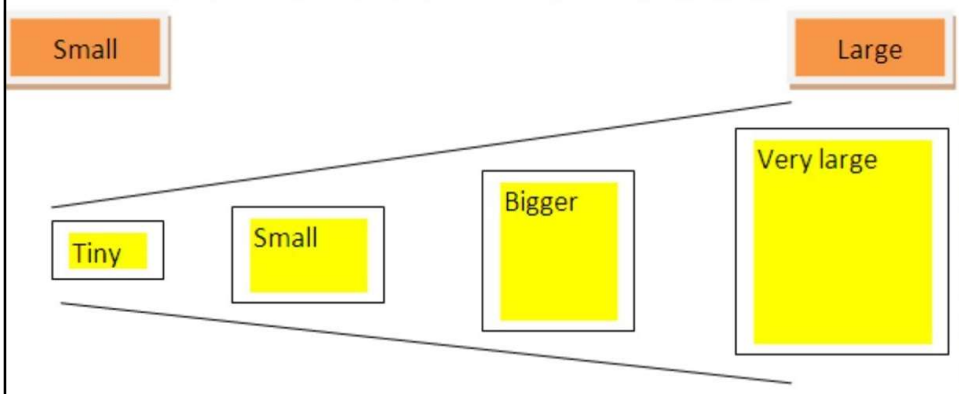


- Allows to a more clear separation of size of work to be done

220

Agile User Stories Estimation

- Affinity Estimating



221

Agile User Stories Estimation

- Affinity Estimating

1. Place two posts on the Wall (small and large).
2. Team members will be asked to come to the Wall with a subset of the product backlog.
3. Team members will size each item relatively to the others on the wall.
   - Silent step.
4. Product owner/Stackeholders present only to provide clarity on some issue.

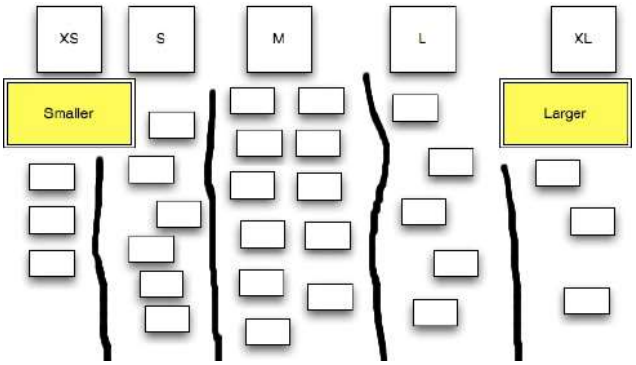
Agile User Stories Estimation

- Affinity Estimating

5. Place all the items on the Wall.
6. After that, all members would look to the Wall and validate if they agree on the estimation.
   5. They may change the place of the user story after some debate.
7. Estimation ends when no more user stories are changed in the Wall.

Agile User Stories Estimation

8. Create columns
   - Relative buckets.
   - T-shirt or fibonacci sequence



224

Agile User Stories Estimation

- Planning Poker
  1. Normally taking place at the iteration planning meeting.
  2. With all the development team (PO might be present to clarify some issue)
  3. Each member have a deck of cards (normally with fibonnaci sequence)
  4. All members must understand the story that needs to be estimated.
  5. Each member assign the story points that thinks is related with the user story.

225

Agile User Stories Estimation

- Planning Poker
  6. High and low numbers explain their point-of-view.
  7. After discussion team play again the cards to try to have consensos..
  8. Repeat the process till the estimates converge to a single estimate that can be used for the story. The number of rounds of estimation may vary from one user story to another.

What about if there is no agreement?

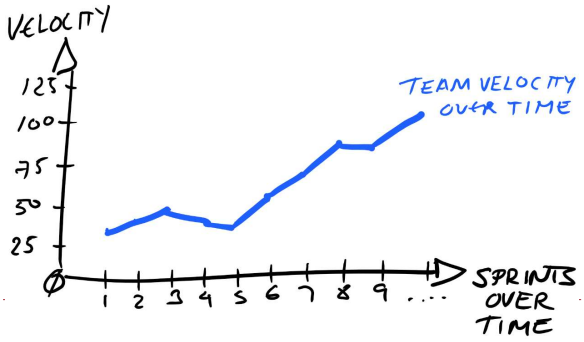
Agile User Stories Estimation

- Planning Poker



Agile Metrics - Velocity

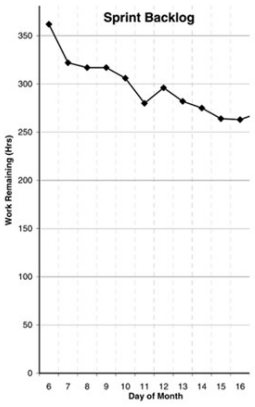
- Number of story Points (or ideal days) that were finished over several sprints.
- Gives an estimation of how much work a team can produce by iteration.

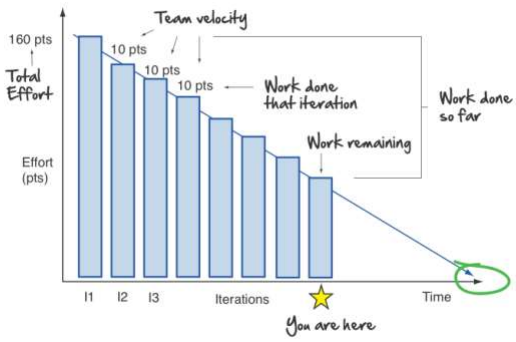


228

Agile Metrics - Burndown Chart

- Summary of estimated task hours remaining for all the tasks (iteration or until end of project).

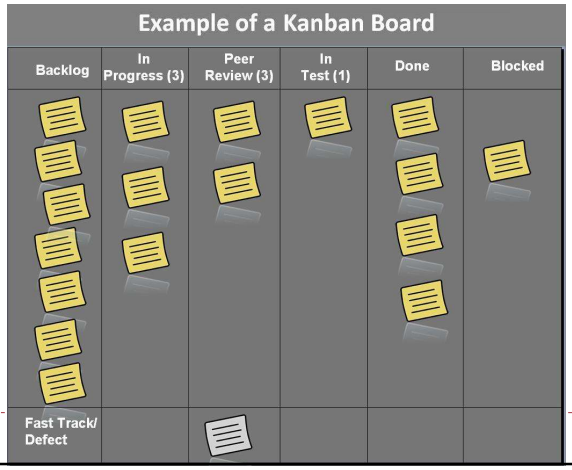




229

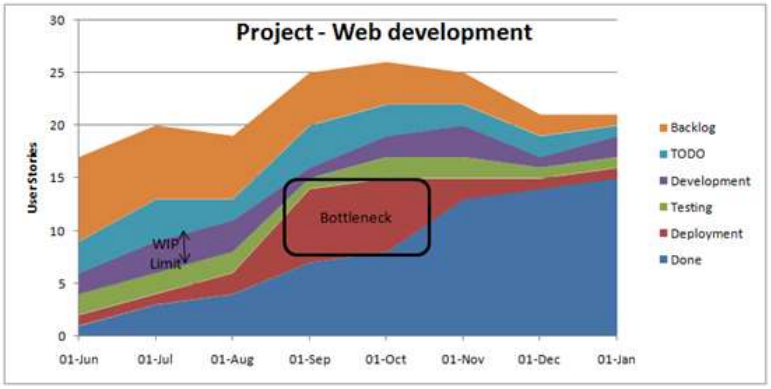
Agile Metrics – Kanban Board

- Gives a visual general overview of task status.
- Allows to detect visually bottlenecks.



Agile Metrics – Cumulative flow diagram

- Allows to understand the flow of work in diferente stages during the project life cycle.



Bibliography

- **Larman, Craig.** *Agile and Iterative Development: A Manager's Guide.* Boston: Addison-Wesley, 2004.(Chapter 1, 2, 3)
- https://www.researchgate.net/publication/331157722_Real_Time_Agile_Metrics_for_Measuring_Team_Performance

- Some figures from the slides were taken from this book